

What I did for P1 (train ontology)

Just writing down what I did at each stage, roughly in the order I did it. Some of this took a few tries to get right so I'm including the mistakes too, not just the final version.

A.2 - building the ontology

I split this into two files: P1_Task1.ttl for the TBox (classes like Service, Equipment, TrainTrip, Train, TrainStation, plus the properties usesEquipment, isRunBy, startStation, endStation) and P1_Task2.ttl for the actual data - two stations (Moss and Stabekk), a train (Train1) with 8 coaches, and one trip (Trip1) going from Moss to Stabekk.

Took me a while to get the two files matching up properly. I had typos like TrainStaion in one file and TrainStation in the other, so they were technically two completely different classes as far as the reasoner was concerned. Same thing happened with the station individuals - I had them typed in one place but then used a differently-spelled version in the actual trip data, so the station used in Trip1 was never actually typed as anything. Fixed all of that once I noticed it.

After that I wrote a SPARQL query to answer the actual question: what time do trains get to Stabekk, and how many coaches. Query joins endStation + arrivalTime + isRunBy/hasCoach and groups by trip so it counts coaches per trip instead of mixing everything together.

Adding OWL restrictions (rules 1-3)

Just having domain/range on the properties doesn't actually force anything to exist - it only says what type something has to be IF it's there. So I added proper restrictions instead:

- Every TrainTrip has to be run by at least one Train.
- Every TrainTrip has to have a startStation and an endStation.
- If something is Equipment and it runs a TrainTrip, it has to be a Train. For this one I had to add an inverse property (runs, inverse of isRunBy) and write it as an intersection class.

To actually check rule 3 worked, I made a test individual called Train2 that was only typed as Equipment (not Train) and gave it a runs relation to Trip1. Ran the Hermit reasoner and Train2 showed up as an inferred member of Train, which is exactly what should happen if the rule is working. Deleted Train2 after testing since it's not real data.

Genus-differentia definitions

Wrote three Aristotelian-style definitions and attached them with skos:definition:

- TrainTrip = a Service that is run by a train and has an origin and destination.
- Train = a piece of Equipment used to run train trips.
- TrainStation = a Location where trains depart from / arrive at.

TrainStation didn't have a superclass before this, so I added a new class called Location to be its genus.

A.5 - stops at a given station

The question here was about trains stopping at a station like As stasjon, and what their destination is. The tricky part is that "stopping at" a station could mean either that it's the origin or the destination of the trip - so I added a new station (AsStation, with a label "As stasjon" so it's readable) and two trips: one starting there (going to Moss) and one ending there (coming from Stabekk), so I'd have both cases to test against.

Query uses UNION for the two cases - if the trip starts at the station I use departureTime, if it ends there I use arrivalTime, and either way I report the trip's endStation as the destination. Ran it and got both trips back correctly, which shows the ontology can actually answer this question.

JSON-LD version of the ABox

Made a JSON-LD file with just the ABox from A.2 (not the As stasjon stuff added later). Checked it in the JSON-LD Playground - looked at the Expanded view to make sure the short names were resolving to the right full IRIs and that things like startStation/hasCoach were being treated as links to other nodes and not just plain text.

Tried loading it straight into Protege and it wouldn't open - turns out this is a known bug in Protege where JSON-LD files using @graph at the top level just fail. Fixed it by converting the file to the fully "expanded" JSON-LD form instead (no @context, no @graph, just a flat array with full IRIs everywhere), and that loaded fine. Checked all 11 individuals were there with the right properties.

One thing I noticed: since this file has no TBox, Protege shows things like arrivalTime and endStation as plain annotations instead of proper object/data properties, because it has nothing telling it what type they're supposed to be. Makes sense since the task specifically wanted ABox only.

End result

Final ontology is P1_Task1.ttl (TBox + restrictions + definitions) importing P1_Task2.ttl (ABox), so together they have both the TBox and ABox as required. The JSON-LD file is kept separate and only has the ABox, per the task.